

拾穗者

我时常梦见一片无边无际的田野。田里长的不是庄稼，是词语。它们有的饱满如麦穗，像“爱”“时间”“土地”这样沉甸甸的，弯着腰；有的干瘪细碎，像“的”“了”“在”，被风一吹就散落一地。我手里提着一只旧布囊，在田里慢慢地走，看见中意的词语，就弯腰拾起来，放进囊中。走完一整片田，我便在田埂上坐下来，把布囊里的东西倒出来点数，想知道这片田野到底盛产哪些词语。

这便是我写这个小程序的全部缘由。它没有任何高深的算法，只是一个在词语田野里低头走路的老人。它的正式名字叫“拾穗者”，其实不过是一个词频统计工具：你给它一篇文字，它还你一张词语的丰收清单。然而于我而言，那些代码片段就是拾穗的步骤，田野、囊、点数的动作，都实实在在地长在程序里。

1 词语的穗子

在田野里，每个词语都是一株独立的穗子。我需要一个容器来暂时存放它们——当然也就是一个链表，每个结点存着词语本身和它出现的次数。我们把它叫“穗子”，学名不妨叫 `WordNode`。

④ 穗子的形状 #

1

```
typedef struct word_node {
    char *word;          /* 词语本身 */
    int count;           /* 被拾起的次数 */
    struct word_node *next;
} WordNode;
=> 主程序: 拾穗的过程 <5>
```

田野广阔，我不会一直盯着同一株穗子。每遇到一个词语，我就得在布囊里翻找一下，看看是不是已经拾起过。布囊就是整个链表，我们用 `find_word` 这个动作来完成翻找。

④ 在布囊中翻找 #

2

```
WordNode *find_word(WordNode *head, const char *word) {
    WordNode *p = head;
    while (p) {
        if (strcmp(p->word, word) == 0)
            return p;
        p = p->next;
    }
    return NULL;
}
=> 主程序: 拾穗的过程 <5>
```

如果布囊里还没有这株穗子，我就得新添一个结点，把它放进去。这个动作叫 `add_word`，它总是把新穗子放在链表的开头，因为刚拾起的总是最新鲜的。

④ 添加新的穗子 #

3

```
WordNode *add_word(WordNode *head, const char *word) {
    WordNode *node = malloc(sizeof(WordNode));
    node->word = strdup(word);
    node->count = 1;
    node->next = head;
    return node;
}
```

```
}
```

=> 主程序: 拾穗的过程 <5>

2 分割词语的落叶

田野里的文字不是天然就分成一株一株穗子的。它是一片连绵的文本，我需要一个分割的动作，把词语从句子中分离出来。这个动作不复杂：我只关心字母和数字，其余的一概视为分隔符。这一步，仿佛是用一把竹耙将落叶聚拢，只留下有用的叶片。get_word 函数每次从输入里扒出一个词，填进我递过去的小竹篮。

@ 扒出一个词 #

4

```
int get_word(FILE *fp, char *buf, int buf_size) {
    int c;
    /* 跳过非字母数字字符 */
    while ((c = fgetc(fp)) != EOF) {
        if (isalnum(c))
            break;
    }
    if (c == EOF) return 0;

    int i = 0;
    buf[i++] = tolower(c);
    while (i < buf_size - 1 && (c = fgetc(fp)) != EOF && isalnum(c)) {
        buf[i++] = tolower(c);
    }
    buf[i] = '\0';
    return 1;
}
```

=> 主程序: 拾穗的过程 <5>

3 走在田野里

现在，一切准备就绪。我打开田野的大门——也许就是一个文件，也许是标准输入，那便是另一片更广阔的田地。我拎着空空的布囊，一步一步走进去，每遇到一个词，就拾起来放进囊中。走完全部田地，我便坐在田埂上清点。

主程序逻辑极其简单，像一段默默行路时光。它所需的外部世界（那些头文件）早已在“外景”里备好，此刻只需轻轻引出。

@ 主程序: 拾穗的过程 #

5

外景 @ <6>

穗子的形状 @ <1>

在布囊中翻找 @ <2>

添加新的穗子 @ <3>

扒出一个词 @ <4>

```
int main(int argc, char *argv[]) {
    FILE *fp = stdin;
    if (argc > 1) {
        fp = fopen(argv[1], "r");
    }
}
```

```

        if (!fp) {
            fprintf(stderr, "无法打开田野 %s\n", argv[1]);
            return 1;
        }
    }

    WordNode *head = NULL;
    char buf[256];

    while (get_word(fp, buf, sizeof(buf))) {
        WordNode *node = find_word(head, buf);
        if (node) {
            node->count++;
        } else {
            head = add_word(head, buf);
        }
    }

    if (fp != stdin) fclose(fp);

    /* 清点: 找出出现次数最多的若干词语 */
    int top_n = 20;
    WordNode *p;
    for (int i = 0; i < top_n && head; i++) {
        WordNode *max_node = head;
        for (p = head->next; p; p = p->next) {
            if (p->count > max_node->count)
                max_node = p;
        }
        if (max_node->count == 0) break;
        printf("%s: %d\n", max_node->word, max_node->count);
        max_node->count = 0;
    }

    /* 释放布囊, 离开田野 */
    while (head) {
        WordNode *next = head->next;
        free(head->word);
        free(head);
        head = next;
    }

    return 0;
}

```

你也许注意到了，我的清点方式很笨拙：每次循环都遍历整个链表，找出当前最高的一个，然后把它标记为零，再找下一个。这就像在田埂上把穗子一遍遍摊开，每次只挑出最饱满的那一株。我完全可以用更精巧的方法，但一个老人坐在田埂上，慢慢地挑拣，本身也是拾穗的一部分。程序只要能用，便不必总是聪明绝顶。

4 外景：必须包含的头文件

每个程序都得有一些外来的支撑，就像我脚下这片田野，总得承认天空和远方山脉的存在。这些头文件正是我的田野之外的外景。

```
@ 外景 #  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <ctype.h>  
=> 主程序：拾穗的过程 <5>
```

6

它们的名字连在一起，看起来像一扇窄窄的栅栏门。推开它，词语田野就在眼前了。

5 运行拾穗者

把上面的代码片段提取出来，编译，就得到了一个叫 `gleaner` 的小东西。你可以这样用它：

```
$ gleaner my_text.txt
```

它会输出文章里出现最多的二十个词，就像老人把布囊里的穗子摊开给你看。如果什么文件也不给，它就安静地等着你从键盘上播种——等你敲完，按下 `Ctrl+D`，它便照样清点你刚才亲手撒下的词语。

对我来说，这便是一种诚实的文学编程了：叙事和代码谁也不为谁服务，它们本就是同一种劳作的两个面向——拾穗的动作与拾穗的故事。代码片段就是那些穗子，而文字则是拾穗人晚归时哼唱的歌。